UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

SEDE SANTO DOMINGO DE LOS TSÁCHILAS

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

ASIGNATURA:

Modelos Discretos para la Ingeniería

TEMA:

Proyecto Unidad 3

Árboles, Grafos y Autómatas Finitos aplicados al Sistema FastBite Delivery

DOCENTE:

Ing. Javier Cevallos

ESTUDIANTES:

Darlin Demera

Jeffry Vera

NIVEL — NRC:

Tercero — 29532

PERÍODO:

S-I ABR 26 – AGO 26

FECHA DE ENTREGA:

28/07/2026

TABLA DE CONTENIDOS

TABLA DE CONTENIDOS 2
1. INTRODUCCIÓN 3
2. OBJETIVOS 3
2.1 Objetivo General 3
2.2 Objetivos Específicos 3
3. DESCRIPCIÓN DEL PROBLEMA O CASO APLICADO 3
4. TEMAS DE ESTRUCTURAS DISCRETAS UTILIZADOS 4
4.1 Árboles Binarios de Búsqueda (BST) 4
4.2 Grafos Ponderados y Algoritmo de Dijkstra 5
4.3 Autómatas Finitos Deterministas (AFD) 5
5. DESARROLLO DEL SISTEMA — DESCRIPCIÓN PASO A PASO 6
5.1 Descripción General 6
5.2 Aplicación del Árbol — CategoryTree (Menú de Productos) 6
5.3 Aplicación del Grafo — DeliveryGraph (Ruta y Tarifa de Envío) 6
5.4 Aplicación del Autómata Finito — Navegación y Autenticación 7
6. EJEMPLOS RESUELTOS 9
6.1 Ejemplo 1 — Árbol BST: Inserción y Recorrido In-Order 9
6.2 Ejemplo 2 — Grafo: Ruta Más Corta con Dijkstra 9
6.3 Ejemplo 3 — Autómata: Secuencia de Eventos de Autenticación 9
7. CONCLUSIONES 11
8. RECOMENDACIONES 11
9. REFERENCIAS 11
ANEXOS 11

1. INTRODUCCIÓN

El desarrollo de software confiable no solo requiere que un programa funcione correctamente, sino que también organice y procese su información mediante estructuras discretas adecuadas al problema que resuelve. Las matemáticas discretas proveen tres herramientas fundamentales para el diseño de sistemas: los árboles, los grafos y los autómatas finitos, cada uno orientado a modelar un tipo distinto de comportamiento: jerarquías y búsquedas ordenadas, redes con costos y rutas, y secuencias de estados válidos dentro de un proceso.

En el presente trabajo se analizan estos tres temas a partir de un caso aplicado real: FastBite Delivery System, una aplicación de escritorio en Java (Swing) que simula un sistema de pedidos y entregas a domicilio. El proyecto integra un Árbol Binario de Búsqueda (BST) para organizar el menú de productos, un Grafo Ponderado con el algoritmo de Dijkstra para calcular la ruta más corta y el costo de envío dentro de una red urbana, y un flujo de control de navegación y autenticación que puede modelarse formalmente como un Autómata Finito Determinista (AFD).

El objetivo de este análisis es identificar, dentro del código fuente del sistema, la aplicación concreta de cada estructura discreta, formalizarla mediante su definición matemática correspondiente (nodos y recorridos, matriz/lista de adyacencia y peso, o quíntupla de estados y transiciones) y verificar su comportamiento mediante ejemplos resueltos.

2. OBJETIVOS

2.1 Objetivo General

Analizar la aplicación práctica de árboles, grafos y autómatas finitos en el proyecto FastBite Delivery System, formalizando cada estructura mediante su definición matemática y verificando su funcionamiento a través de ejemplos resueltos basados en el código fuente del sistema.

2.2 Objetivos Específicos

- Identificar los módulos del sistema FastBite en los que se implementan estructuras de árboles, grafos y autómatas finitos.
- Describir formalmente el Árbol Binario de Búsqueda utilizado para organizar y filtrar el menú de productos por categoría.
- Describir formalmente el Grafo Ponderado y el algoritmo de Dijkstra empleados para calcular la ruta de entrega más corta y la tarifa asociada.
- Modelar el flujo de navegación y autenticación del sistema como un Autómata Finito Determinista, definiendo sus estados, alfabeto y función de transición.
- Resolver ejemplos prácticos que evidencien el comportamiento de cada estructura ante distintas entradas.
- Concluir sobre la importancia de aplicar estructuras discretas adecuadas al diseño de sistemas de software confiables y escalables.

3. DESCRIPCIÓN DEL PROBLEMA O CASO APLICADO

FastBite Delivery System es una aplicación de escritorio desarrollada en Java 17 con interfaz Swing (tema oscuro FlatLaf) y gestionada con Maven. Simula el flujo completo de un servicio de comida rápida: un cliente navega un menú organizado por categorías, arma un carrito de compras, selecciona su ubicación dentro de una red vial urbana y el sistema calcula automáticamente la ruta más corta desde el restaurante hasta el punto de entrega, junto con el costo del envío. Adicionalmente, cuenta con un panel administrativo protegido por autenticación, desde donde se gestionan el menú y la red de reparto.

Para que el sistema resuelva estos requerimientos de forma eficiente y ordenada, se apoya en tres estructuras discretas distintas, resumidas en la Tabla 1.

Tabla 1. Estructuras discretas identificadas en el proyecto

#	Estructura	Módulo / Clase	Problema que resuelve
1	Árbol Binario de Búsqueda (BST)	com.fastbite.ds.CategoryTree	Organizar el menú por categoría/nombre y permitir recorridos ordenados y filtrado.
2	Grafo Ponderado + Dijkstra	com.fastbite.ds.DeliveryGraph	Calcular la ruta más corta y el costo de envío entre el restaurante y el cliente.
3	Autómata Finito Determinista (AFD)	com.fastbite.ui.MainFrame / AdminLoginDialog	Controlar de forma válida la navegación entre vistas y el proceso de autenticación del administrador.

La ausencia de una estructura adecuada en cualquiera de estos tres puntos tendría consecuencias directas: sin un árbol ordenado, filtrar el menú requeriría recorrer la lista completa en cada búsqueda; sin un grafo con Dijkstra, no sería posible garantizar la ruta de menor costo entre múltiples caminos posibles; y sin un modelo de estados claro, el sistema podría permitir transiciones inválidas, como acceder al panel de administrador sin autenticarse.

4. TEMAS DE ESTRUCTURAS DISCRETAS UTILIZADOS

4.1 Árboles Binarios de Búsqueda (BST)

Un árbol binario de búsqueda es una estructura jerárquica en la que cada nodo tiene como máximo dos hijos (izquierdo y derecho), y se cumple la propiedad de orden: todo valor en el subárbol izquierdo es menor que el del nodo, y todo valor en el subárbol derecho es mayor o igual. Esta propiedad permite realizar inserciones, búsquedas y recorridos ordenados de forma eficiente.

Tabla 2. Características del BST

Característica	Descripción
Criterio de orden	Categoría del producto; en caso de empate, nombre del producto
Recorrido usado	In-order (izquierda → nodo → derecha)
Resultado del recorrido	Lista de productos ordenada alfabéticamente por categoría y nombre
Complejidad promedio	O(log n) en inserción/búsqueda; O(n) en el peor caso (árbol degenerado)

4.2 Grafos Ponderados y Algoritmo de Dijkstra

Un grafo ponderado G = (V, E, w) está formado por un conjunto de vértices V, un conjunto de aristas E que conectan pares de vértices, y una función de peso w que asigna un costo (en este caso, distancia en metros) a cada arista. El algoritmo de Dijkstra encuentra la ruta de menor costo acumulado desde un vértice origen hacia todos los demás vértices, siempre que los pesos sean no negativos.

Tabla 3. Características del grafo de FastBite

Característica	Descripción
Tipo de grafo	No dirigido, ponderado, conexo
Vértices (V)	20 nodos: 1 restaurante central + 19 sectores de la ciudad
Peso de las aristas	Distancia vial aproximada en metros
Estructura interna	Lista de adyacencia (HashMap<String, List<RoadEdge>>)
Algoritmo de ruta mínima	Dijkstra con cola de prioridad (PriorityQueue / min-heap)
Uso del resultado	Distancia total → cálculo de tarifa: tarifaBase + distancia × costoPorMetro

4.3 Autómatas Finitos Deterministas (AFD)

Un autómata finito determinista se define formalmente como una quíntupla $M = (Q, \Sigma, \delta, q_o, F)$, donde Q es un conjunto finito de estados, Σ es el alfabeto de entradas (eventos), $\delta: Q \times \Sigma \rightarrow Q$ es la función de transición, $q_o \in Q$ es el estado inicial y $F \subseteq Q$ es el conjunto de estados de aceptación. Un AFD garantiza que, para cada estado y cada evento válido, exista una única transición definida, lo que evita comportamientos ambiguos o inválidos en el sistema.

En FastBite, este modelo describe con precisión dos comportamientos del sistema que en el código se controlan mediante banderas booleanas (isAdminLoggedIn, authenticated) y un CardLayout: (a) la navegación entre la vista de Cliente y la vista de Administrador, y (b) el proceso de autenticación dentro del diálogo de login.

5. DESARROLLO DEL SISTEMA — DESCRIPCIÓN PASO A PASO

5.1 Descripción General

El sistema FastBite Delivery está construido en tres paquetes principales: com.fastbite.model (entidades: LocationNode, RoadEdge, MenuItem, OrderItem, OrderState), com.fastbite.ds (estructuras de datos: CategoryTree, DeliveryGraph, DijkstraResult) y com.fastbite.ui (interfaz gráfica Swing). A continuación se describe, paso a paso, cómo se aplica cada estructura discreta dentro del flujo real del sistema.

5.2 Aplicación del Árbol — CategoryTree (Menú de Productos)

Funcionamiento paso a paso:

- 1. Al iniciar la aplicación, CategoryTree carga un menú por defecto (11 productos) mediante inserciones sucesivas.
- 2. Cada inserción compara primero por categoría y, en caso de empate, por nombre del producto (insertRecursive).
- 3. Si el valor comparado es menor, el nodo se inserta en el subárbol izquierdo; en caso contrario, en el derecho.
- 4. Para mostrar el menú, se ejecuta un recorrido in-order (inOrderTraversal), que devuelve los productos ordenados.
- 5. El filtrado por categoría (getItemsByCategory) reutiliza este recorrido y aplica un filtro adicional sobre el resultado.

Figura 1. Extracto de CategoryTree.java — inserción según orden BST

```
int comp = item.getCategory().compareTo(current.item.getCategory());
if (comp == 0) {
    comp = item.getName().compareTo(current.item.getName());
}
if (comp < 0) {
    current.left = insertRecursive(current.left, item);
} else {
    current.right = insertRecursive(current.right, item);
}
```

5.3 Aplicación del Grafo — DeliveryGraph (Ruta y Tarifa de Envío)

Funcionamiento paso a paso:

- 1. El grafo se inicializa con 20 nodos fijos (initializeSampleNetwork), representando el restaurante y sectores de la ciudad, y sus conexiones viales con distancias en metros.
- 2. Cuando el cliente selecciona su ubicación de entrega, se invoca findShortestPath(destinationNodeId).
- 3. Dijkstra inicializa todas las distancias en infinito, salvo el nodo origen (el restaurante) en 0, y usa una cola de prioridad para explorar siempre el nodo no visitado más cercano.
- 4. Por cada vecino, si la nueva distancia acumulada es menor que la registrada, se actualiza la distancia y se guarda el nodo predecesor.

- 5. Al llegar al nodo destino (o vaciar la cola), se reconstruye la ruta completa recorriendo los predecesores desde el destino hasta el origen.
- 6. La distancia total obtenida se usa para calcular la tarifa: tarifaBase + (distancia × costoPorMetro).

Figura 2. Extracto de DeliveryGraph.java — núcleo de Dijkstra

```
while (!pq.isEmpty()) {
    NodeDistance current = pq.poll();
    if (current.distance > distances.get(current.nodeId)) continue;
    if (current.nodeId.equals(destinationNodeId)) break;
    for (RoadEdge edge : adjList.get(current.nodeId)) {
        double newDist = distances.get(current.nodeId) + edge.getDistanceMeters();
        if (newDist < distances.get(edge.getTarget().getId())) {
            distances.put(edge.getTarget().getId(), newDist);
            previousNodes.put(edge.getTarget().getId(), current.nodeId);
            pq.add(new NodeDistance(edge.getTarget().getId(), newDist));
        }
    }
}
```

5.4 Aplicación del Autómata Finito — Navegación y Autenticación

El comportamiento de MainFrame ante los botones de navegación y el resultado del AdminLoginDialog puede formalizarse como un AFD con los siguientes componentes:

Tabla 4. Definición formal del autómata de navegación/autenticación

Componente	Definición en FastBite
Estados (Q)	{VISTA_CLIENTE, VISTA_ADMIN, DIALOGO_LOGIN}
Alfabeto (Σ)	{clic_menu_cliente, clic_panel_admin, login_exitoso, login_fallido/cancelado, clic_salir_admin}
Estado inicial (q_0)	VISTA_CLIENTE
Estados de aceptación (F)	{VISTA_CLIENTE, VISTA_ADMIN} (estados estables de la interfaz)
Función de transición (δ)	Ver Tabla 5

Tabla 5. Tabla de transiciones δ(estado, evento) → estado

Estado actual	Evento	Estado siguiente
VISTA_CLIENTE	clic_panel_admin (no autenticado)	DIALOGO_LOGIN
DIALOGO_LOGIN	login_exitoso	VISTA_ADMIN
DIALOGO_LOGIN	login_fallido / cancelado	VISTA_CLIENTE
VISTA_ADMIN	clic_menu_cliente	VISTA_CLIENTE
VISTA_ADMIN	clic_salir_admin	VISTA_CLIENTE
VISTA_CLIENTE	clic_panel_admin (ya autenticado)	VISTA_ADMIN

Este modelo evidencia una regla clave del sistema: no existe ninguna transición directa desde VISTA_CLIENTE hacia VISTA_ADMIN que no pase por el estado DIALOGO_LOGIN con el evento login_exitoso, salvo que la bandera isAdminLoggedIn ya sea verdadera en la sesión actual. Es decir, el AFD hace explícita —y verificable— una restricción de seguridad que en el código está implícita en un condicional.

Figura 3. Extracto de MainFrame.java — transición controlada por el evento de login

```
private void handleAdminAccess() {
    if (isAdminLoggedIn) {
        showView("ADMIN");
    } else {
        AdminLoginDialog loginDlg = new AdminLoginDialog(this);
        loginDlg.setVisible(true);
        if (loginDlg.isAuthenticated()) {
            isAdminLoggedIn = true;
            showView("ADMIN");
        }
    }
}
```

6. EJEMPLOS RESUELTOS

6.1 Ejemplo 1 — Árbol BST: Inserción y Recorrido In-Order

Problema: dado el siguiente subconjunto de productos, determinar el árbol resultante y su recorrido in-order.

Tabla 6. Productos de entrada (orden de inserción)

Orden	Categoría	Nombre
1	Hamburguesas	Burger Suprema Doble
2	Pizzas	Pizza Pepperoni Deluxe
3	Bebidas	Coca Cola Zero 500ml
4	Hamburguesas	Burger Crispy Chicken

Solución: comparando por categoría (Bebidas < Hamburguesas < Pizzas alfabéticamente), el árbol se construye con "Burger Suprema Doble" como raíz. "Coca Cola Zero" (Bebidas) va al subárbol izquierdo; "Pizza Pepperoni Deluxe" (Pizzas) va al subárbol derecho. "Burger Crispy Chicken" comparte categoría con la raíz, por lo que se compara por nombre ("Crispy" < "Suprema") y se ubica en el subárbol izquierdo de la raíz.

Recorrido in-order resultante: Coca Cola Zero 500ml → Burger Crispy Chicken → Burger Suprema Doble → Pizza Pepperoni Deluxe. El resultado confirma que el árbol entrega los productos agrupados y ordenados por categoría, sin necesidad de un paso de ordenamiento adicional.

6.2 Ejemplo 2 — Grafo: Ruta Más Corta con Dijkstra

Problema: calcular la ruta más corta desde NODE_REST (restaurante) hasta ESTADIO, usando el grafo definido en el sistema.

Tabla 7. Aristas relevantes para el ejemplo (fragmento del grafo)

Origen	Destino	Peso (m)
NODE_REST	ESTADIO	950
NODE_REST	HOSPITAL_CENTRAL	1200
HOSPITAL_CENTRAL	ESTADIO	800

Solución: Dijkstra evalúa dos caminos posibles hacia ESTADIO: (a) directo NODE_REST → ESTADIO = 950 m; (b) NODE_REST → HOSPITAL_CENTRAL → ESTADIO = 1200 + 800 = 2000 m. Al comparar ambos, el algoritmo conserva la distancia mínima (950 m) y descarta la ruta más larga.

Con costoPorMetro = 0.003 y tarifaBase = 1.50, la tarifa de envío es: 1.50 + (950 × 0.003) = 1.50 + 2.85 = $4.35.

6.3 Ejemplo 3 — Autómata: Secuencia de Eventos de Autenticación

Problema: determinar el estado final del sistema ante la secuencia de eventos: clic_panel_admin, login_fallido, clic_panel_admin, login_exitoso, clic_menu_cliente.

Tabla 8. Traza de ejecución del autómata

Paso	Estado actual	Evento	Estado resultante
1	VISTA_CLIENTE	clic_panel_admin	DIALOGO_LOGIN
2	DIALOGO_LOGIN	login_fallido	VISTA_CLIENTE
3	VISTA_CLIENTE	clic_panel_admin	DIALOGO_LOGIN
4	DIALOGO_LOGIN	login_exitoso	VISTA_ADMIN
5	VISTA_ADMIN	clic_menu_cliente	VISTA_CLIENTE

Resultado: el sistema termina en el estado VISTA_CLIENTE. La traza confirma que un intento fallido de login no produce ninguna transición hacia VISTA_ADMIN, y que solo el evento login_exitoso habilita dicho acceso, validando el comportamiento esperado del autómata.

7. CONCLUSIONES

- El Árbol Binario de Búsqueda implementado en CategoryTree permite mantener el menú de FastBite ordenado por categoría y nombre sin necesidad de reordenarlo en cada consulta, ya que el recorrido in-order entrega directamente los productos en el orden requerido.
- El Grafo Ponderado junto con el algoritmo de Dijkstra resuelve de forma óptima el problema de calcular la ruta y el costo de entrega más bajos entre múltiples caminos posibles dentro de la red urbana simulada, evidenciando la utilidad de las estructuras de grafos en problemas de logística.
- El flujo de navegación y autenticación del sistema, aunque implementado con banderas booleanas y condicionales en el código, corresponde formalmente a un Autómata Finito Determinista; modelarlo como tal permite verificar de manera explícita que no existen transiciones inválidas hacia el panel administrativo sin pasar por una autenticación exitosa.
- El análisis conjunto de estas tres estructuras evidencia que un mismo sistema de software puede requerir distintos modelos discretos según el tipo de problema que resuelve cada módulo: jerarquías ordenadas, redes con costo, o procesos con estados válidos.

8. RECOMENDACIONES

- Documentar explícitamente, dentro del código fuente, la relación entre las banderas de control (isAdminLoggedIn, authenticated) y el modelo de autómata al que corresponden, para facilitar el mantenimiento del sistema.
- Evaluar el balanceo del árbol (por ejemplo, mediante un AVL o Red-Black Tree) si el catálogo de productos llegara a crecer considerablemente, para evitar la degradación a O(n) en árboles desbalanceados.
- Ampliar la red del grafo con datos reales de distancias (GPS/GIS) en lugar de coordenadas aproximadas, y considerar variantes de Dijkstra con restricciones de tráfico o capacidad vial.
- Formalizar otros flujos del sistema (por ejemplo, el ciclo de vida de un pedido: carrito → confirmación → en camino → entregado) como autómatas finitos adicionales, ya que presentan la misma estructura de estados y transiciones válidas.

9. REFERENCIAS

• Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

• Rosen, K. H. (2019). Discrete Mathematics and Its Applications (8th ed.). McGraw-Hill Education.

• Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Pearson.

• Weiss, M. A. (2014). Data Structures and Algorithm Analysis in Java (3rd ed.). Pearson Education.

• Oracle Java Documentation. Consultado en 2026. Documentación oficial de Java utilizada como referencia técnica.

ANEXOS

Anexo A. Código fuente completo de CategoryTree.java, DeliveryGraph.java, MainFrame.java y AdminLoginDialog.java.

Anexo B. Diagrama del árbol BST del menú (capturas del sistema en ejecución).

Anexo C. Diagrama de la red de 20 nodos del grafo urbano y ruta resaltada por Dijkstra.

Anexo D. Diagrama de estados del autómata de navegación/autenticación.